

1. Halaman Judul

Praktikum Struktur Data - Modul 5: STACK

2. Tujuan Praktikum

- Mempraktikkan cara membuat Stack murni dan mengeksekusi operasi Pop, Push, serta Peek.
- Merancang simulasi program menu interaktif dan model tumpukan benda fisik.
- Mampu menghitung kalkulasi *size* pada tumpukan secara efisien.

3. Dasar Teori

Karena prinsip kerjanya hanya terpusat pada satu pintu masuk dan keluar, komputasi Stack dikategorikan memiliki kompleksitas konstan atau $O(1)$. Ini berarti kecepatan pemrosesan data akan selalu konsisten dan tidak melambat meskipun jumlah datanya sangat banyak, menjadikannya pilihan struktur data yang sangat optimal untuk fungsi-fungsi memori *cache* jangka pendek.

4. Program Python & Algoritma

Berikut adalah kode penyelesaian komprehensif untuk Tugas 1 sampai dengan Tugas 6. Untuk simulasi antarmuka menu di Tugas 4, digunakan otomatisasi input agar mempermudah pembuatan file PDF laporan.

```
1 print("===== TUGAS LEVEL MUDAH =====")
2
3 print("\n--- Tugas 1 & 2: Stack Sederhana & Push/Pop ---")
4 tumpukan_tugas = []
5
6 # Proses Push
7 tumpukan_tugas.append("File A")
8 tumpukan_tugas.append("File B")
9 tumpukan_tugas.append("File C")
10 print("Status Stack sesudah Push:", tumpukan_tugas)
11
12 # Proses Pop
13 file_ditarik = tumpukan_tugas.pop()
14 print(f"Objek yang dikeluarkan: {file_ditarik}")
15 print("Status Stack sekarang:", tumpukan_tugas)
16
17
18 print("\n--- Tugas 3: Cek Elemen Paling Atas (Peek) ---")
19 if len(tumpukan_tugas) > 0:
20     paling_atas = tumpukan_tugas[-1]
21     print(f>Data di ujung atas adalah: {paling_atas}")
22 else:
23     print("Tidak ada data, Stack kosong.")
24
25
26 print("\n===== TUGAS LEVEL MENENGAH =====")
27
28 print("\n--- Tugas 4 & 5: Program Menu Interaktif & Hitung Size ---")
29 sistem_stack = []
30
31 # List ini hanya untuk mensimulasikan pilihan user agar kode bisa dieksekusi sekali jalan
32 # Kode aksi -> 1: Push, 2: Pop, 3: Ukuran/Size, 4: Selesai
33 simulasi_user = [
34     (1, "Dokumen Rahasia"),
35     (1, "Laporan Keuangan"),
36     (3, ""),
37     (2, ""),
38     (4, "")
39 ]
40
41 print("=== Menjalankan Sistem Interaktif ===")
42 for aksi, nilai in simulasi_user:
43     if aksi == 1:
44         sistem_stack.append(nilai)
45         print(f"[Aksi Push] Menambahkan {nilai} ke tumpukan.")
46     elif aksi == 2:
47         if len(sistem_stack) > 0:
48             hapus = sistem_stack.pop()
49             print(f"[Aksi Pop] Data '{hapus}' ditarik dari tumpukan.")
50         else:
51             print("[Aksi Pop] Gagal dilakukan. Tumpukan habis!")
52     elif aksi == 3:
53         ukuran = len(sistem_stack)
54         print(f"[Aksi Size] Total objek saat ini berjumlah: {ukuran} elemen.")
55     elif aksi == 4:
56         print("[Aksi Keluar] Menutup sistem...")
57         break
```

```

58
59
60 print("\n--- Tugas 6: Pemodelan Tumpukan Piring ---")
61 rak_piring = []
62
63 print("Ibu mencuci dan menumpuk piring di rak...")
64 rak_piring.append("Piring Keramik")
65 rak_piring.append("Piring Melamin")
66 rak_piring.append("Piring Kaca")
67 print("Kondisi rak:", rak_piring)
68
69 # Mengambil piring untuk makan malam
70 piring_makan = rak_piring.pop()
71 print(f"\nAdik mengambil {piring_makan} untuk makan malam.")
72 print(f"Piring bersih yang tersisa: {rak_piring}")

```

===== TUGAS LEVEL MUDAH =====

```

--- Tugas 1 & 2: Stack Sederhana & Push/Pop ---
Status Stack sesudah Push: ['File A', 'File B', 'File C']
Objek yang dikeluarkan: File C
Status Stack sekarang: ['File A', 'File B']

```

```

--- Tugas 3: Cek Elemen Paling Atas (Peek) ---
Data di ujung atas adalah: File B

```

===== TUGAS LEVEL MENENGAH =====

```

--- Tugas 4 & 5: Program Menu Interaktif & Hitung Size ---
=== Menjalankan Sistem Interaktif ===
[Aksi Push] Menambahkan Dokumen Rahasia ke tumpukan.
[Aksi Push] Menambahkan Laporan Keuangan ke tumpukan.
[Aksi Size] Total objek saat ini berjumlah: 2 elemen.
[Aksi Pop] Data 'Laporan Keuangan' ditarik dari tumpukan.
[Aksi Keluar] Menutup sistem...

```

```

--- Tugas 6: Pemodelan Tumpukan Piring ---
Ibu mencuci dan menumpuk piring di rak...
Kondisi rak: ['Piring Keramik', 'Piring Melamin', 'Piring Kaca']

```

```

Adik mengambil Piring Kaca untuk makan malam.
Piring bersih yang tersisa: ['Piring Keramik', 'Piring Melamin']

```

5. Analisis Hasil

- Praktikum memvalidasi bahwa setiap kali kita memanggil (`append()`), data lama akan tertimbun oleh data baru.
- Pada menu interaktif, terlihat keharusan untuk memasukkan validasi panjang list (`(len(stack) > 0)`). Jika hal ini diabaikan dan *user* mencoba melakukan *Pop* pada Stack yang belum diisi (atau sudah habis), program akan langsung berhenti paksa (Crash/Error).

6. Jawaban Diskusi

1. **Apa yang dimaksud dengan Stack?** Sebuah struktur organisasi data linear yang bekerja selayaknya tabung sempit, di mana akses keluar masuknya hanya bisa melalui satu ujung terbuka saja.
2. **Mengapa Stack menggunakan prinsip LIFO?** Karakteristiknya yang hanya punya satu pintu akses (TOP) mengakibatkan benda apa pun yang disisipkan paling akhir pasti berada persis di depan pintu akses tersebut, sehingga ia yang paling siap untuk dikeluarkan lebih dulu.
3. **Apa perbedaan Push dan Pop?** Kedua operasi ini berkebalikan; Push bertugas melampirkan entitas baru di atas tumpukan, sementara Pop bertugas mencabut entitas teratas tersebut hingga tumpukan berkurang.
4. **Mengapa operasi Stack memiliki kompleksitas $O(1)$?** Karena target lokasinya selalu tetap di indeks ujung. Program tidak perlu repot-repot menyisir (traversal) atau menggeser elemen-elemen di bagian bawah tumpukan.
5. **Apa fungsi Call Stack?** Menyimpan jejak historis pemanggilan fungsi di memori komputer, memastikan komputer tidak tersesat dan tahu harus mengembalikan hasil pemrosesan (return) ke fungsi awal pemanggilnya.
6. **Apa hubungan Stack dengan Rekursi?** Setiap fungsi rekursif yang dieksekusi akan ditumpuk satu per satu ke dalam Call Stack secara otomatis oleh *compiler* komputer hingga fungsi tersebut menemukan kondisi akhir penyelesaiannya.
7. **Apa yang menyebabkan Stack Overflow?** Kapasitas memori tabung Stack sistem operasi komputer mencapai batas maksimal dan tumpah, biasanya akibat logika fungsi rekursif berulang tanpa henti (infinity loop).
8. **Mengapa fitur Undo menggunakan Stack?** Karena saat membatalkan kesalahan ketik, kita ingin mengembalikan dokumen tepat ke 1 detik sebelumnya (langkah terakhir), bukan langsung mereset teks dari ketikan pertama. Logika memutar balik waktu selangkah demi selangkah ini hanya bisa ditangani oleh sistem LIFO.
9. **Apa kelebihan dan kekurangan Stack?** Unggul dalam hal efisiensi dan kecepatan manajemen memori instan. Namun sangat lemah dalam fleksibilitas pencarian, karena kita tidak diizinkan mengubah elemen di dasar tumpukan tanpa menghancurkan tumpukan di atasnya.

10. Penerapan Stack di dunia nyata:

- Menyusun riwayat History pada peramban internet.
- Fungsi Undo/Redo pada peranti lunak desain dan dokumen.
- Memeriksa keseimbangan kurung penutup oleh *Compiler* program.
- Fitur transisi navigasi "Kembali" (*Back button*) pada HP cerdas.
- Tata kelola urutan *Function Call* memori komputer.